CHAPTER 13

Software Security

ACRONYMS

CC	Common Criteria
SDLC	Secure Development Life Cycle

INTRODUCTION

Security has become a significant issue in software development because of potential misuse and increasing malicious activity targeting computer systems. In addition to the usual correctness and reliability concerns, software developers must pay attention to the security of the software they develop. Secure software development builds security by following a set of established and/or recommended rules and practices. Secure software maintenance complements secure software development by ensuring that no security problems are introduced during software maintenance and that identified vulnerabilities, which are errors that attackers can exploit, can be handled during the software life cycle. Security vulnerabilities are not only introduced at the development, but also by third party components such as libraries, COTS, or OS.

BREAKDOWN OF TOPICS FOR SOFTWARE SECURITY

Breakdown of topics for the Software Security KA is shown in Figure 13.1.

1. Software Security Fundamentals [37, 9]

A generally accepted belief about software security is that it is much better to design security into software than to patch it in after the software is developed. To design security into software, one must consider every development life cycle stage. Secure software development involves software requirements security, software design security, software construction security and software testing security. In addition, security must be considered during software maintenance, as security faults and loopholes can be and often are introduced during maintenance.

1.1. Software Security [10]

Security is a product quality characteristic representing the degree to which a product or system protects information and data so that persons or other products or systems have data access appropriate to their types and levels of authorization [10]. (For more information about product quality, refer to the Software Quality KA.)

1.2. Information Security [11]

Information security preserves confidentiality, integrity and availability of information. Other properties, such as authenticity, accountability, non-repudiation and reliability can also be involved [11]. *Confidentiality* is the property of ensuring that information is not disclosed to unauthorized individuals, entities or processes. *Integrity* is the property of accuracy and completeness. *Availability* is the property of being accessible and usable on demand by an authorized entity. Software engineers should define the security properties of their software and maintain them throughout the software development life cycle.

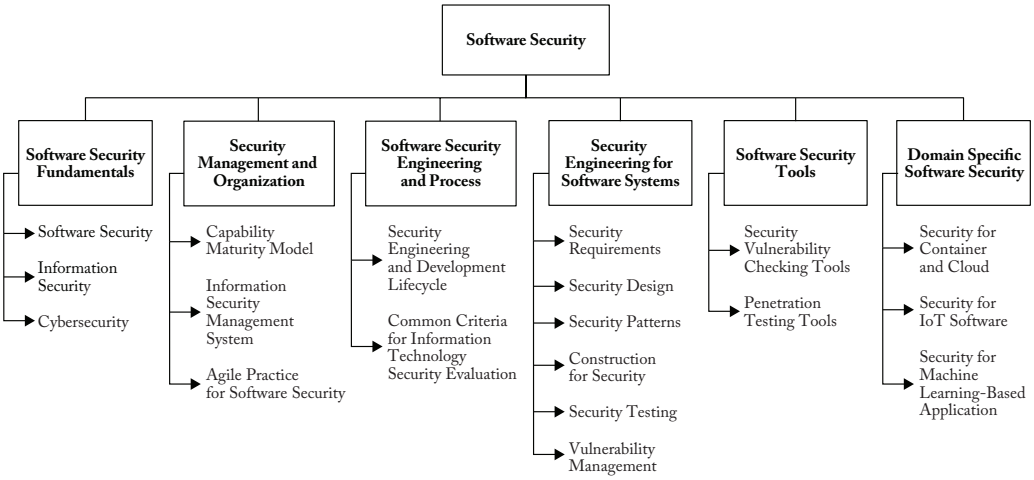

Figure 13.1. Breakdown of Topics for the Software Security KA

1.3. Cybersecurity [12][38]

Cybersecurity is safeguarding of people, society, organizations and nations from cyber risks. Safeguarding means to keep cyber risk at a tolerable level.

Generally, cybersecurity addresses security issues in cyberspace, including the following:

- Social engineering attacks
- Hacking
- Malicious software (*malware*)
- Other potentially unwanted software [12]

Software engineers should consider the mitigation of such threats as part of software development.

2. Security Management and Organization [1*, c7][13]

Security governance and management are most effective when they are systematic; in other words, when they are woven into the culture and fabric of organizational behaviors and actions. Project managers need to elevate software security from a stand-alone technical concern to an enterprise issue [1].

2.1. Capability Maturity Model [3*, c22][14]

Many organizations practice security engineering in the development of computer programs, including operating systems, functions that manage and enforce security, packaged software products, middleware, and applications. Therefore, a diverse array of individuals must know how to apply appropriate methods and practices, including product developers, service providers, system integrators, system administrators and even security specialists. Systems Security Engineering — Capability Maturity Model (SSE-CMM), which helps measure the process capability of an organization that performs risk assessments [14], can be an important tool.

2.2. Information Security Management System [15]

ISO/IEC 27001:2022 specifies the requirements for establishing, implementing, maintaining and continually improving an information security management system (ISMS) within the organizational context [15]. ISMS is a documented plan for managing the technology-related security of an

organization. This includes documenting risks and taking measures to address them, aiming to protect the organization's data and prevent security breaches [15]. Organizations should use it to continually conduct risk assessments to identify security risks and vulnerabilities and implement protective measures by deploying an IT team to monitor these risks. An ISMS can thus also raise new or changed existing software security requirements. In addition, software security requirements are derived from laws, regulations and obligations for compliance.

2.3. Agile Practice for Software Security [4,c15,c16]

Agile teams need to understand and adopt security practices and take more responsibility for their systems' security. Security professionals must learn to accept change, work faster and more iteratively, and think about security risks and how to manage risks in incremental terms. Finally, and most important, security needs to become an enabler instead of a blocker. The keys to a successful Agile security program are the involvement of the security team and developers, enablement, automation, and agility to keep up with Agile teams [4].

3. Software Security Engineering and Processes

3.1. Security Engineering and Secure Development Life Cycle (SDLC) [1*,c1][16][36]

Software is only as secure as its development process. Security must be built into software engineering to ensure software security. The SDLC concept is one trend that aims to do this. SDLC uses a classical spiral model that views security holistically from the perspective of the software life cycle and ensures that security is inherent in software design and development, not an afterthought later in production. The SDLC process is claimed to reduce software maintenance costs and increase software reliability against security-related faults.

Recently, DevSecOps (meaning the integration of development, security and operations) has emerged. Beyond SDLC, DevSecOps includes an approach to culture, automation and platform design to make the software life cycle as Agile and responsible as Agile development and continuous integration (CI).

3.2. Common Criteria for Information Technology Security Evaluation [3*, c22, c25][34][35]

Security evaluation establishes confidence in the security functionality of IT products and the assurance measures applied to them. The evaluation results may help consumers determine whether IT products meet their security needs or standards conformity. ISO/IEC 15408:2022, named Common Criteria (CC) for Information Technology Security Evaluation, is useful as a guide for developing, evaluating and/or procuring IT products with security functionality [34].

CC addresses the protection of assets from unauthorized disclosure, modification or loss of use. The categories of protection relating to these three types of security failure are commonly called *confidentiality*, *integrity* and *availability*, respectively.

4. Security Engineering for Software Systems [1*,c1,c3][3*,c1,c3]

4.1. Security Requirements [1*,c3][2,c2][3*,c20,c30][18]

Security requirements engineering includes elicitation, specification, and prioritization. It considers threats, as illustrated by misuse and abuse cases, threat actors, security risk assessments, selection and application of specification methods, prioritization methods, inspections, and revisions. Selection of lifecycle models may impact the order of activities, and software product revision implies a need to revisit security requirements. Traceability of security requirements throughout the development process is important, and security teams may include specialists in security

requirements. Numerous methods and tools exist in support of security requirements engineering.

4.2. Security Design [1*,c4][2,c5][3*,c20,c31][17,40]

Security design concerns how to prevent unauthorized disclosure, creation, change, deletion or denial of access to information and other resources. It also concerns how to tolerate security-related attacks or violations by limiting damage, continuing service, speeding repair and recovery, and failing and recovering securely. Access control is a fundamental concept of security. Most controls build on cryptographic algorithms and cryptographic material like keys. It is important to carefully select these and how cryptographic material is created, distributed and managed.

Software design security deals with the design of software modules that fit together to meet the security objectives specified in the security requirements. To meet security requirements, developers conduct threat modeling, illustrating how a system is being attacked to specify a security design for the mitigation. This step clarifies the details of security considerations and develops the specific steps for implementation. Factors considered may include frameworks and access modes that set up the overall security monitoring/enforcement strategies, as well as the individual policy enforcement mechanisms.

4.3. Security Patterns [1*,c4][19][20, 21]

A *security pattern* describes a particular recurring security problem that arises in a specific context and presents a well-proven generic solution [21].

4.4. Construction for Security [1*,c5][3*,c20,c31][22, 23, 24]

Software construction security concerns how to write programming code for specific situations to address security considerations. The term software construction security can mean different things to different people. It can mean the way a specific function is coded so that the code itself is secure, or it can mean the coding of security into software. Unfortunately, most people entangle the two meanings without distinction. One reason for such confusion is that it is unclear how to ensure a specific coding is secure. For example, in the C programming language, the expressions "i<<1" (shift the binary representation of i's value to the left by one bit) and "2*i" (multiply the value of variable i by constant 2) mean the same thing semantically, but do they have the same security ramifications?

The answer could be different for different combinations of ISAs and compilers. Because of this lack of understanding, software construction security — in its current state — mostly refers to the second aspect mentioned above: the coding of security into software. Coding of security into the software can be achieved by following recommended rules. A few such rules are:

- Structure the process so that all sections requiring extra privileges are modules. The modules should be as small as possible and perform only the tasks that require those privileges.
- Ensure that any assumptions in the program are validated. If this is not possible, document them for the installers and maintainers so they know the assumptions attackers will try to invalidate.
- Ensure that the program does not share objects in memory with any other program.
- Check every function's error status. Do not recover unless neither the error's cause nor its effects affect any security considerations. The program should restore the state of the software to the state it had before the process began and then terminate.

Although there are no bulletproof ways to achieve secure software development, some general guidelines exist that can be helpful.

These guidelines span every phase of the software development life cycle. The Computer Emergency Response Team (CERT/CC) publishes reputable guidelines [22], and the following are its top 10 software security practices:

1. Validate input.
2. Heed compiler warnings.
3. Architect and design for security policies.
4. Keep it simple.
5. Default deny.
6. Adhere to the principle of least privilege.
7. Sanitize data sent to other software.
8. Practice defense in depth.
9. Use effective quality assurance techniques.
10. Adopt a software construction security standard.

4.5. Security Testing

[1*,c5][2,c7][3*,c24,c31][26, 27]

Security testing ensures that the implemented software meets the security requirements. It also verifies that the software implementation contains none of the known vulnerabilities. Whereas general software testing methods can handle the former, the latter requires security-specific testing methods. (For more information about testing, refer to the Software Testing KA.)

There are two general approaches to security-specific testing. The first approach includes detecting vulnerabilities through static analysis, which can be conducted on the source code or compiled binaries. A static analysis on the source code can be used to detect programming language or implementation-specific vulnerabilities, while static analysis on compiled binaries can be used to detect vulnerabilities that are not apparent in the source code due to compiler optimizations or hidden in the compiled third-party components. Static analysis can be automated using tools, however while automation can play a significant role, the expertise of security professionals are required to properly operate and configure the tools, and verify the results.

The other approach to detect vulnerabilities is through dynamic testing, typically using techniques such as vulnerability assessment or penetration testing (also known as the ethical hacking test), to detect vulnerabilities in software behavior. Like static analysis, there are tools that can automate dynamic testing, such as web application scanners and fuzzing tools. Security experts skilled in the application domain should be engaged to perform these tests, and such tests should always be conducted within legal boundaries and with proper authorization. The latter aspects are crucial to differentiate such tests from illegal hacking activities.

4.6. Vulnerability Management

[1*,c5][3*,c24][28,29, 30]

Using sound coding practices can help substantially reduce software defects commonly introduced during implementation [1]. Such common security defects are categorized and shared with databases: the Common Vulnerabilities and Exposures (CVE) [28], Common Weakness Enumeration (CWE) [29], and Common Attack Pattern Enumeration and Classification (CAPEC) [30]; Common Vulnerability Scoring System (CVSS) [41] expresses characteristics and severity of software vulnerabilities. Programmers can refer to these databases for security implementation, and some tools are available to check common vulnerabilities in code. Security maintenance encompasses the task to mitigate effects of vulnerabilities in a system and third party components which the system uses. The task often comes with a vulnerability disclosure process that allows to report the identification of vulnerabilities.

5. Software Security Tools

5.1. Security Vulnerability Checking Tools

[1*,c6][25]

Security vulnerability checking tools, such as source code analyzers and binary analysis tools, can be used to identify potential

security vulnerabilities and issues. Source code analyzers scrutinize code to detect security vulnerabilities, such as injection flaws, buffer overflows, and insecure library use. They are useful at finding vulnerabilities that can be identified through code patterns and logical flaws. Binary analysis tools, on the other hand, examine compiled code, including third-party libraries, for vulnerabilities that might not be apparent in the source code or that arise from the compilation process. While these tools significantly aid in detecting vulnerabilities, they cannot find all vulnerabilities. For example, they might not capture vulnerabilities that manifest in hard-to-produce software states or that crop up in unusual circumstances [1].

5.2. Penetration Testing Tools [2,c4]

Penetration testing tools can be used to evaluate a system's security in its operational environment. These tools perform controlled attacks on the system to uncover vulnerabilities and security weaknesses, using techniques such as fuzzing [2], where malformed, malicious, or random data is submitted to the system's various entry points to detect faults. The use of penetration testing tools to expose vulnerabilities provide insights into how an actual attacker could exploit the system.

6. Domain-Specific Software Security

6.1. Security for Container and Cloud [31*, c1-c3]]

Cloud infrastructure and services are often inexpensive and easy to provision, which can quickly lead to having many assets strewn all over the world and forgotten. These forgotten assets are like a ticking time bomb, waiting to explode into a security incident [31].

One important difference with cloud environments is that physical assets and protection are generally not a concern. Developers can outsource asset tags, anti-tailgating, slab-to-slab barriers, placement of data center windows, cameras, and other physical security and physical asset tracking controls [31].

6.2. Security for IoT Software [32,33]

As part of today's internet of things (IoT), systems are interconnected with many other devices, especially back-end systems suffering from all the well-known security flaws inherent in today's business IT. Attackers gaining access to business IT platforms, for instance, by exploiting browser vulnerabilities, will likely also gain access to weakly protected IoT industrial devices. This can cause severe damage, including safety incidents. Hence, the introduction of a massive number of end points from the consumer or industrial environment creates fertile ground for the exploitation of weak links. Hardening these end points, securing device-to-device communications, and ensuring device and information credibility in what until now have been closed, homogeneous systems present new challenges. Comprehensive risk and threat analysis methods, as well as management tools for IoT platforms, are required [33].

6.3. Security for Machine Learning–Based Application [39,c8]

Although machine learning techniques are widely used in many systems, machine learning presents a specific vulnerability. Attackers can change the decisions of machine learning models. There are two kinds of attacks: model poisoning, which attacks training data, and evasion, which attacks inputs to trained models [39].

MATRIX OF TOPICS VS. REFERENCE MATERIAL

Topic	Allen et al. 2008 [1*]	Bishop 2019 [3*]	Dotson 2019 [31*]
1. Software Security Fundamentals			
1.1. Software Security			
1.2. Information Security			
1.3. Cybersecurity		c23	
2. Security Management and Organization	c7		
2.1. Capability Maturity Model		c22	
2.2. Information Security Management System			
2.3. Agile Practice for Software Security			
3. Software Security Engineering and Processes			
3.1. Security Engineering and Secure Development Life Cycle	c1		
3.2. Common Criteria for Information Technology Security Evaluation		c22, c25	
4. Security Engineering for Software Systems	c1, c3	c1, c3	
4.1. Security Requirements	c3	c20, c31	
4.2. Security Design	c4	c20, c31	
4.3. Security Patterns	c4		
4.4. Construction for Security	c5	c20, c31	
4.5. Security Testing	c5	c24, c31	
4.6. Vulnerability Management		c24	
5. Software Security Tools			
5.1. Security Vulnerability Checking Tools	c6		
5.2. Penetration Testing Tools	c4	c31	
6. Domain-Specific Software Security			
6.1. Security for Container and Cloud			c1-c3
6.2. Security for IoT Software			
6.3. Security for Machine Learning-Based Application			

FURTHER READINGS

J. Viega, *Building Secure Software: How to Avoid Security Problems the Right Way*, Addison-Wesley, 2011.

This book introduces the definition of Software Security and the activities to develop and maintain secure software. It includes not only the software development process but also the related

activities such as auditing and the monitoring of service.

L. Kohnfelder, *Designing Secure Software: A Guide for Developers*, No Starch Press, 2021.

This book describes security activities in the software design and implementation phases, including secure programming and web security. It also introduces best practices for secure software development.

C.W. Axelrod, *Engineering Safe and Secure Software Systems*, Artech House Publishers, 2012.

This book describes engineering activities to make software systems safe and secure from a risk management viewpoint. It introduces risk assessment and mitigation methods for security and safety.

REFERENCES

[1*] J.H. Allen, S.J. Barnum, R.J. Ellison, G. McGraw, and N.R. Mead, *Software Security Engineering: A Guide for Project Managers*, Addison-Wesley Professional, 2008.

[2] G. McGraw, *Software Security: Building Security In*, Addison-Wesley Professional, 2006.

[3] M. Bishop, *Computer Security*, 2nd Edition, Addison-Wesley Professional, 2019.

[4] L. Bell, M. Brunton-Spall, R. Smith, and J. Bird, *Agile Application Security*, O'Reilly, 2017.

[5] T. Hsiang-Chih Hsu, *Hands-On Security in DevOps: Ensure continuous security, deployment, and delivery with DevSecOps*, Packt Publishing, 2018.

[6] T. Hsiang-Chih Hsu, *Practical Security Automation and Testing: Tools and techniques for automated security scanning and testing in DevSecOps*, Packt Publishing, 2019.

[7] G. Wilson, *DevSecOps: A leader's guide to producing secure software without compromising flow, feedback and continuous improvement*, Rethink Press, 2020.

[8] L. Rice, *Container Security: Fundamental Technology Concepts That Protect Containerized Applications*, O'Reilly & Associates Inc., 2020.

[9] ISO/IEC/JTC1 SC27 Standards: Trustworthiness, Cryptography, Data security, Cryptography, Security evaluation and testing, Security control, Identity management and privacy technologies.

[10] ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model.

[11] ISO/IEC 27000:2018 Information technology — Security techniques — Information security management systems — Overview and vocabulary.

[12] ISO/IEC 27032:2012 Information technology — Security techniques — Guidelines for cybersecurity.

[13] ISO/IEC 19770-1:2017 Information technology — IT asset management — Part 1: IT asset management systems — Requirements.

[14] ISO/IEC 21827:2008 Information technology — Security techniques — Systems Security Engineering — Capability Maturity Model (SSE-CMM).

[15] ISO/IEC 27001:2022 Information

security, cybersecurity and privacy protection — Information security management systems — Requirements.

[16] M. Howard and S. Lipner, *The Security Development Lifecycle*, Microsoft Press, 2006.

[17] F. Swiderski and W. Snyder, *Threat Modeling: Design for Security*, Wiley, 2014.

[18] D. Firesmith, "Security use cases," *Journal of Object Technology*, Vol. 2, No. 1, pp. 53-64, 2003.

[19] E. Fernandez-Buglioni, *Security Patterns in Practice: Designing Secure Architectures Using Software Patterns*, Wiley, 2013.

[20] C. Nagappan, R. Lai, and R. Steel, *Core Security Patterns: Best Practices and Strategies for J2EE, Web Services, and Identity Management*, Prentice Hall, 2005.

[21] M. Schumacher, E. Fernandez-Buglioni, D. Hybertson, F. Buschmann, and P. Sommerlad, *Security Patterns: Integrating Security and Systems Engineering*, Wiley, 2006.

[22] R.C. Seacord, *The CERT C Secure Coding Standard*, Addison-Wesley Professional, 2008.

[23] R.C. Seacord, *Secure Coding in C and C++*, Addison-Wesley Professional, 2013.

[24] D. Long, F. Mohindra, D. Seacord, R.C. Sutherland, and D.F. Svoboda, *The CERT Oracle Secure Coding Standard for Java*, 2011.

[25] J. Erickson, *Hacking: The Art of Exploitation*, 2nd Edition, No Starch Press, 2008.

[26] K. Scarfone, M. Souppaya, A. Cody, and A. Orebaugh, *Technical Guide to Information Security Testing and Assessment*, NIST SP800-115, 2008.

[27] PCI Security Standards Council, PCI DSS: Payment Card Industry Data Security Standard, Version 3.2, 2017.

[28] *MITRE*, "Common Vulnerabilities and Exposures (CVE)," https://www.cve.org/.

[29] *MITRE*, "Common Weakness Enumeration (CWE)," https://cwe.mitre.org/.

[30] *MITRE*, "Common Attack Pattern Enumeration and Classification (CAPEC)," https://capec.mitre.org/.

[31*] C. Dotson, *Practical Cloud Security*, O'Reilly, 2019.

[32] "Internet of Things Security Best Practices," *IEEE*, 2017, https://standards.ieee.org/wp-content/uploads/import/documents/other/whitepaper-internet-of-things-2017-dh-v1.pdf.

[33] "IoT 2020: Smart and secure IoT platform," *IEC*, 2016, https://www.iec.ch/basecamp/iot-2020-smart-and-secure-iot-platform.

[34] ISO/IEC 15408-1:2022 Information security, cybersecurity and privacy protection — Evaluation criteria for IT security — Part 1: Introduction and general model.

[35] ISO/IEC 18045:2008 Information technology — Security techniques — Methodology for IT security evaluation.

[36] DoD Enterprise DevSecOps, https://dodcio.defense.gov/Portals/0/Documents/Library/DoD%20Enterprise%20DevSecOps%20Fundamentals%20v2.5.pdf.

[37] C. Easttom, *Computer Security Fundamentals*, 4th Edition, Pearson IT Certification, 2019.

[38] Y. Diogenes and E. Ozkaya, *Cybersecurity — Attack and Defense Strategies*, Second Edition, Packt Publishing, 2019.

[39] C. Chio and D. Freeman, *Machine Learning and Security: Protecting Systems with Data and Algorithms*, O'Reilly, 2018.